DAMS: Double Deep Q-Network Approach for DDoS Attack Mitigation in SDN

Shurok Khozam
SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris
Telecom SudParis
Palaiseau, France
shurok.khozam@telecom-sudparis.eu

Gregory Blanc
SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris
Telecom SudParis
Palaiseau, France
gregory.blanc@telecom-sudparis.eu

Sebastien Tixeuil
LIP6, Sorbonne Université
Sorbonne University
Paris, France
Sebastien.Tixeuil@lip6.fr

Eric Totel
SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris
Telecom SudParis
Palaiseau, France
eric.totel@telecom-sudparis.eu

Abstract

5G represents the most recent and sophisticated generation of mobile communications. It boasts features such as rapid response times, minimal latency, and substantial bandwidth. 5G networks leverage advanced technologies, including Software-Defined Networking (SDN), which plays a pivotal role in designing these networks, enabling network administrators to centralize their control and management. These networks are increasing sharply in size as well as in functionality, especially with the growth of the Internet of Things (IoT), which has given rise to numerous challenges. Nowadays, attackers are developing diverse techniques to exploit vulnerabilities and insufficient security in IoT devices, making them exploitable in large-scale attacks. Consequently, it has become mandatory to define appropriate countermeasures in order to defend these networks against different attack types. Many studies have attempted this using traditional methods, which may be efficient but often prove inadequate or detrimental to legitimate users, highlighting the need for more intelligent solutions. In this article, we use the Double Deep Q-Network (DDQN), a reinforcement learning (RL) algorithm, in conjunction with the programmability of SDN to mitigate DoS attacks in 5G networks. The mitigation process is realized first by supporting remediation selection to maximize the response efficiency while reducing the adverse impact on the network. Second, it involves automating remediation deployment, which minimizes downtime and reduces manual and error-prone incident handling.

CCS Concepts

• **Security and privacy → Malware and its mitigation**; • **Networks → Programmable networks**.



This work is licensed under a Creative Commons Attribution International 4.0 License.

ICFNDS '24, Marakech, Morocco
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1170-1/24/12
https://doi.org/10.1145/3726122.3726282

Keywords

Reinforcement learning, Software defined networks, Distributed denial of service.

ACM Reference Format:
Shurok Khozam, Gregory Blanc, Sebastien Tixeuil, and Eric Totel. 2024. DAMS: Double Deep Q-Network Approach for DDoS Attack Mitigation in SDN. In *The 8th International Conference on Future Networks & Distributed Systems (ICFNDS '24), December 11–12, 2024, Marakech, Morocco.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3726122.3726282

1 Introduction

The deployment of 5G networks has ushered in a new era of high-speed, low-latency connectivity [2, 12]. However, this rising technology also presents new security challenges, as the increasing complexity of the network architectures and the large number of connected devices create new attack vectors for malicious actors, such as exploiting vulnerabilities in IoT devices [11]. 5G networks enable the mutualization of a single infrastructure to provide tailored network services to different verticals thanks to softwarization and virtualization layers, including technologies such as SDN and NFV. These technologies allow for the creation of more dynamic, flexible, efficient, and programmable network architectures [2]. However, these advancements bring new challenges. For instance, distributed denial of service (DDoS) attacks pose significant threats and cause rapid and dynamic changes in network parameters, emphasizing the need for advanced and adaptive security measures as they employ multi-vector techniques and exploit IoT vulnerabilities.

In light of the limitations associated with traditional security tools such as rule-based systems, which often rely on predefined rules and signatures making them vulnerable to new attack methods, we recognized the need for a more adaptive approach to secure SDN. Traditional methods struggle with scalability, as the volume of malicious traffic in modern attacks can overwhelm their infrastructure. Additionally, these methods produce high latency and have slow detection and response times, making them unsuitable for real-time applications [7, 18].

To address these challenges, we employ reinforcement learning (RL) algorithms, which are promising for securing network devices as they can learn from large data volumes, adjust strategies, and adapt to new and unseen attacks in real-time, even from limited network data [14, 19]. This article focuses on securing a softwarized network that relies on an SDN infrastructure and leverages the controller to centralize data collection and policy deployment. This network comprises various devices, including servers and hosts. Typically, hosts access services normally, but they can be exploited to attack servers, leading to service disruption for legitimate users. Our primary goal in this article is to ensure network resilience while maintaining service availability. This goal is achieved by reasoning about the network state to select appropriate countermeasures and enforcing them in the data plane. This is carried out by employing the DDQN algorithm. The model-free RL approach effectively handles high-dimensional state spaces, stabilizes training, and avoids overestimation.

The remainder of this paper is organized as follows. Section 2 offers insights into reinforcement learning techniques and reviews relevant literature, while Section 3 outlines the attacker's model we are considering. Our proposed mitigation strategy and RL model are detailed in Section 4. The experiments, results and comparisons are discussed in Section 5. Finally, Section 6 encapsulates concluding remarks and outlines directions for future work.

2 Background and Related Work

Reinforcement Learning (RL) is a machine learning approach where an agent learns to make decisions by interacting with its environment, aiming to maximize cumulative rewards over time. Key components in RL include states S, actions A, rewards R, and exploration probability ϵ. States represent the current situation, actions are decisions affecting future states, and rewards $R = f(S_{t-1}, S_t)$ provide feedback. The exploration probability ϵ balances exploring new actions and exploiting known ones. The goal is to develop a policy $\pi(s)$ that maximizes the cumulative discounted reward $G_t = \sum_{k=t}^{t+T} \gamma^{(k-t)} \cdot r_{k+1}$, where r_{k+1} is the reward obtained after taking an action at time step k, and γ is the discount factor.

In Q-Learning, the agent, modeled as a Markov Decision Process (MDP), aims to estimate the optimal action-value function $Q(s, a)$ to maximize G_t. Deep Q-Learning (DQN) enhances Q-Learning by using a neural network to approximate $Q(s, a)$, minimizing the temporal difference error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim D} \left[(r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-) - Q(s_t, a_t; \theta))^2 \right].$$

Double DQN (DDQN) further refines this by using two Q-networks to decouple action selection and evaluation, stabilizing training and mitigating overestimation bias, with the target Q-value update:

$$y_t = r_t + \gamma Q(s_{t+1}, \arg\max_{a'} Q(s_{t+1}, a'; \theta); \theta^-).$$

Given that D represents the experience replay buffer, which stores past transitions from which the training samples are drawn, θ and θ^- are the weights of the current and target Q-networks, respectively, and a' denotes the next action selected according to the maximum Q-value estimated by the target Q-network.

2.1 Related work

Many approaches focused on providing security solutions and valuable insights into attack detection and mitigation tools in SDN and 5G network environments. Within this context, a system referred to as LSTM-FUZZY [15] was developed to detect and mitigate DDoS and Portscan attacks in SDN environments. This system combines Long Short-Term Memory (LSTM) and fuzzy Logic methodologies and operates through three phases: characterization, anomaly detection, and mitigation. Evaluations in two scenarios involving IP flows from SDN Floodlight controllers and the CICDDoS 2019 dataset demonstrate the system's efficiency in network management and successful detection and mitigation of attacks in SDN environments [15]. However, this method faced challenges, including overblocking legitimate traffic, issues with LSTM's interpretability, training complexity, and fuzzy logic's rule design.

Within the RL framework, an approach integrating DQN and Proximal Policy Optimization (PPO) algorithms was introduced to mitigate three types of attacks in SDN infected by Mirai-like malware, including SSH server port scanning with brute force login, DNS tunneling for data exfiltration, and a Slowloris-based application layer DDoS attack [23]. This approach involved classifying incoming packets and flows as malicious or normal, redirecting malicious packets to virtual appliances for disabling and blocking malicious users and services. However, the scalability of this framework is limited for larger networks.

Wang et al. [21] proposed a strategy that employs a hybrid deep learning method in the detection phase and an IP address-based traceback mitigation method in the mitigation phase to detect and mitigate DDoS attacks. Liu et al. [13] used the Deep Deterministic Policy Gradient (DDPG) algorithm to intelligently defend against various DDoS flooding attacks, including TCP SYN, UDP, and ICMP flooding. Implemented within a Mininet environment equipped with a Ryu controller, the DDPG algorithm dynamically adjusts its response to effectively throttle malicious traffic. The results show that the DDPG-based agent can successfully mitigate DDoS flooding attacks across different protocols. Bandhana et al. employed a MDP for modeling an APT attacker [4], where the attacker's objective is to exfiltrate data from a specific server within a local network to a C&C server on the Internet. The defender, embodied in all clients and servers, has a non-null detection probability.

Yungaicela-Naula et al. [22] focused on addressing slow-rate DDoS attacks with a flexible architecture employing an intrusion detection system trained with machine learning models, a DRL-based mitigation module for real-time prediction of optimal strategies, and a network monitoring module that provides feedback to adjust the DRL-based module's actions.

3 Attacker model

The concentration of research efforts to tackle DDoS attacks is justified by their long history since the end of the 1990s and the fact that they remain a substantial threat [20]. Notably, the escalation is quantified by a substantial increase in the average daily DDoS attacks, rising to 29.3% attacks per day in the fourth quarter of 2022. This escalation marks a noteworthy 3.5-fold surge from the 8.4 attacks per day recorded at the end of 2021 [16]. Moreover, DDoS attackers use new techniques, such as multi-vector flooding,

to improve their attacks.
In our approach, we assume that the attacker targets a host or a network segment with the highest bandwidth to launch its attacks on the server, aiming to maximize disruption of service availability. By targeting the highest bandwidth, we are considering the worst-case scenario to ensure robust defenses against potential threats. The attacker is capable of executing multiple attack types from a compromised host: (1) *ICMP flooding* to overwhelm a system and cause it to crash or become inaccessible [3, 5]; (2) *TCP-SYN flooding* (which distribution reached 26.7% in the second quarter of 2023, making it one of the top 3 most prevalent attacks in that period [9]) where the attacker does not complete the TCP connection establishment, leaving the connection half-open and consuming the target device's resources; (3) *UDP flooding*, where the attacker typically spoofs the source IP addresses of the UDP packets to make it difficult for the target to identify the actual source of the attack (this makes it challenging to block the attack traffic based on the source IP address [8]).

The attacker proceeds by using a substantial quantity of high-priority threads. Each of those opens its own connection to the server and executes the predefined attack in autonomy in order to achieve the highest impact. The duration of each attack thread varies highly and is randomly defined. On the other hand, the duration of the entire attack is controlled by our experiment.

4 Mitigation Strategy

4.1 Mitigation Process

A mitigation process typically involves multiple steps, including network state modeling and automated countermeasure selection (state and actions in section 4.2), dynamic mapping of high-level strategies (RL actions) into low-level security configurations (using open virtual switch (OVS) commands), and rigorous validation of implemented measures (via the reward mechanism presented in section 4.2), with the aim of ensuring network performance for normal users. Additionally, our proposal for a mitigation strategy includes various standalone technical components communicating with each other, as depicted in Fig. 1. These components are divided into *the Data plane, the Control plane, and the Application Plane*. To facilitate communication between components in our framework, we developed several APIs, where the controller gets the observations regarding the data plane through its southbound interfaces and sends its commands through northbound interfaces to the application plane. Our SDN network encompasses both honest users and an attacker trying to compromise the availability of a specific server by transmitting a substantial volume of malicious traffic. This proposal aims to limit the attacker's ability to overwhelm the server by limiting the number of packets it can insert into the network, in other terms, controlling the volume of data it can send. Additionally, we ensure that benign users can continue using the network without problems during the attack. We also consider that intrusion detection is carried out beforehand by a third party, which shares its findings with the system, enabling the ability to distinguish between honest and malicious sources.

The RL state modeling and decision making is running on the application plane. The proposed RL model itself is outlined in Section 4.2. This model involves an RL agent that uses the reward

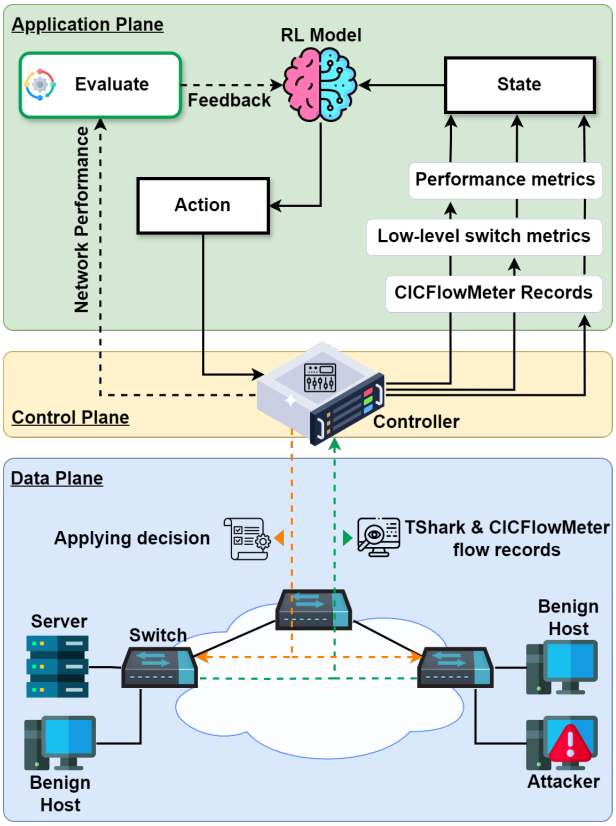


Figure 1: The proposed mitigation system.

derived from various metrics to map the current network status to the most appropriate actions that will improve the situation in the next step. These metrics are calculated based on captured network packets by listening to switches' interfaces governed by the controller for a given amount of time. With this knowledge, and based on its neural network, the agent compares various actions and their probability to affect the current network situation positively. Then, it chooses the one with the highest probability according to the network situation with the aim of ensuring that benign users do not experience delays or data losses (in other words, the most appropriate action leading to the highest reward and best network situation). The controller translates these countermeasures into network decisions deployed on the same governed switches. In Fig. 2 we present the training algorithm from a high-level perspective. While in Algorithm. 1, we present the pseudo-code detailing how the agent is trained. We start by the initialization (defining the network topology and the training environment (`env`)). Then the agent iterates through training episodes, each of which starts by electing server and attacker hosts from available network hosts randomly (via `*_election()` functions on lines 2 and 3). After that, the network starts operating, and an initial state is set. The algorithm then iterates through steps. Each step starts by determining the action to be taken (`determine_action`), then executing this action (with `apply_action_`). To evaluate the impact of the chosen action on the network, a new state is calculated based on malicious

and benign flows that have traversed the monitored

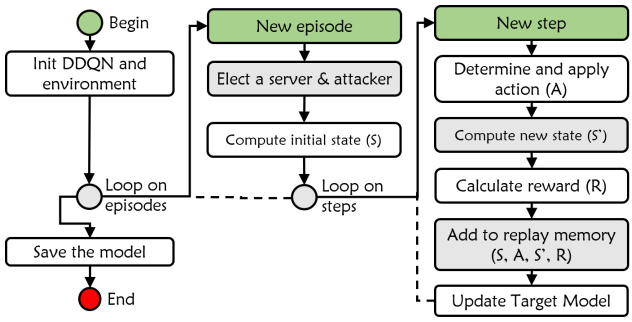


Figure 2: Training Algorithm.

network interfaces over a given period of time (line 9 to line 16). Based on the evolution of the state, consecutive to the enforcement of the action, the reward is calculated (via `env.calc_reward`) as explained later in Section 4.2. The samples describing the evolution of the state before and after taking the action as well as the reward deduced are stored in the agent's replay memory (line 18) till it reaches a certain occupancy rate (line 20), after which the temporary model will start training using these samples (line 21). Reaching either the best situation (*win* state) or the worst situation (*lose* state) stops the training process (line 23), where the best and worst situation are identified when performance metrics surpass specific predefined thresholds. The weights of the target model are updated based on the temporary model every specific number of steps (with this number being determined experimentally), or upon reaching a win/lose state (line 24).

4.2 RL model

In this section, we present our RL model, which is a key component of the training process using the DDQN algorithm. The RL model consists of three components: *state*, *actions*, and *reward*.

State. The selection of variables to represent the environment's state is a crucial aspect of the optimization process as it represents the neural network's input. The size of the input is determined by the product of the number of hosts and the number of state metrics. Subsequently, once the agent has been trained, it utilizes the state representation to make informed decisions and take appropriate actions based on its observation of the environment. In the context of our SDN network, the network state is composed of multiple metrics. The state is thus defined for each host using the metrics presented in Table 1, which are extracted using different methods based on collected information from controlled switches' interfaces. In Table 1, "forward packets" refer to the traffic originating from hosts and directed towards the server. By contrast, "backward packets" denote the server's responses and acknowledgments.

Actions. The agent makes decisions without knowledge of the host's behavior (that is, without knowing whether it is an attacker or a normal user). Subsequently, it evaluates the correctness of its decisions based on their impact on network performance, considering whether they result in an increase or decrease in performance. The

Algorithm 1 Training Algorithm

```
1:  for each episode in env do
2:      servers ← perform_server_election()
3:      attackers ← perform_attacker_election()
4:      start_network(servers, attackers)
5:      current_state ← env.get_initial_state()
6:      for each step in episode do
7:          action ← agent.determine_action(current_state)
8:          env.apply_action_on_controlled_switches(action)
9:          new_state ← env.get_new_state() as
10:             start_benign_flow()
11:             start_malicious_flow()
12:             wait(20s)
13:             stop_malicious_flow()
14:             wait(10s)
15:             stop_benign_flow()
16:             return state
17:         reward ← env.calc_reward(current_state, new_state)
18:         agent.add_to_replay_memory(current_state, action,
            new_state, reward)
19:         current_state ← new_state
20:         if agent.replay_memory.has_enough_experience()
            then
21:             agent.perform_experience_replay()
22:         end if
23:         if is_win_or_lose(new_state) or is_time_to_update( tar-
            get_model) then
24:             agent.update(target_model))
25:         end if
26:     end for
27: end for
```

Table 1: Network state metrics

Feature name	Feature description	Extraction method
Total_pkt_fwd	Total packets in the forward direction	Tshark & CICFlowMeter
Total_pkt_bwd	Total packets in the backward direction	Tshark & CICFlowMeter
Total_size_pkt_fwd	Total size of packets in the forward direction	Tshark & CICFlowMeter
Total_size_pkt_bwd	Total size of packets in the backward direction	Tshark & CICFlowMeter
Nbr_rcv_bytes	Number of received bytes	Controller through OVS commands
Nbr_snt_bytes	Number of transmitted bytes	Controller through OVS commands
Bw	Bandwidth	Controller through OVS commands
Nbr_loss_Pkt	Number of lost packets	Manually computed
Nbr_delivered_pkt	Number of delivered packets	Manually computed
Nbr_sent_bytes_s	Number of transmitted bytes per second	Manually computed
Nbr_sent_pkt_s	Number of transmitted packets per second	Manually computed

actions are: (1) Increase data transfer's capacity for flows coming from a host and passing through controlled switches; (2) Decrease this capacity; and (3) Do nothing.

Reward. The reward function is crucial for evaluating an agent's actions and guiding it toward optimal behaviors. However, defining

this function is challenging, particularly due to the need to adjust environmental parameters. This study presents a reward function that balances network performance and the impact of security measures based on the following state metrics. (1) Transmission time; (2) Number of transmitted bytes per second; and (3) Number of lost packets. Then, (1) and (2) are used to calculate transmission delay. The reward function (R) is defined as:

$$R = \alpha \cdot \text{reward}_1(\text{packet_loss}) + \beta \cdot \text{reward}_2(\text{delay}) \quad (1)$$

The coefficients α and β are determined through experimentation. The reward of packet loss is represented by the following formula:

$$\text{reward}_1(\text{packet_loss}) = 0.02 - \text{avg}(\text{packet_loss } \%) \quad (2)$$

Where $avg(packet_loss\%)$ represents the average packet loss percentage of all normal hosts. The reward for delay is calculated by the following equation:

$$\text{reward}_2(\text{delay}) = 0.01 - \frac{\text{avg}(\text{delay})}{\max(\text{delay})} \quad (3)$$

Where $avg(delay)$ represents the average delay of all normal hosts, $max(delay)$ represents the maximum delay (set to a fixed experimental value depending on network resources). The delay of each host is determined by:

$$\text{delay}(\text{host}) = \frac{\text{transmission_time}}{\text{count}(\text{delivered_bits})} \quad (4)$$

The values of 0.02 for the maximum acceptable loss percentage and 0.01 for the maximum acceptable delay are defined according to our experimental environment and our network architecture.

5 Experiments and results

In this section, we provide an overview of the implementation process while training the agent. Then, we present and analyze the results obtained from our experiments, followed by a qualitative comparison and discussions demonstrating the effectiveness of our approach in mitigating diverse DDoS attack scenarios.

5.1 Training the RL Agent for Effective Mitigation

We first present a mechanism to implement our proposed mitigation strategy (Fig. 1) to counter an attacker capable of generating different types of attacks like ICMP, UDP, and TCP-SYN floods using various tools (e.g., Scapy[1], MHDDoS[2]) within the context of an SDN network emulated using Mininet[3] which introduces real-life behavioral randomness. Benign users are emulated by generating flows using the **DITG tool**[4]. Benign traffic flows last for $d_1 = 30$ seconds ($t_1 = 0s$ to $t_3 = 30s$), whereas malicious traffic flows last for $d_2 = 20$ seconds ($t_1 = 0s$ to $t_2 = 20s$). During these periods, tshark[5] captures network traffic packets by listening on the controlled switches' interfaces – the ones facing the hosts – and stores network information in a PCAP file. Our approach employs a comprehensive network monitoring methodology using CICFlowMeter [6], which extracts the network's features, at the granularity of network flows, from the PCAP file generated by tshark and stores them in a CSV file. The controller further gets state metrics depending on the extracted features and transmits the network's state and performance metrics to the RL agent using several APIs, enabling it to compute the corresponding reward. The RL agent then deduces the action to be taken and pushes it to the controller, which transforms it into OVS commands to be applied on the network. The complete code for this approach is available on a public GitHub repository[6].

In order to train the agent to master a mitigation strategy, a training algorithm based on DDQN iterates through episodes and steps to achieve this objective, where the key tuned variables in the DDQN training algorithm are as follows: epsilon decay is 0.99, discount rate (γ) is 0.85, and learning rate (α) is 0.005. In each episode, we randomly choose an attacker, a server, and benign hosts from the network hosts list $h_i; i \in N$ (number of hosts). For each entity, we assign a random bandwidth ranging from 0.01 to 3.1 Mbps, enabling them to send varied traffic volumes, thus emulating real-life behavior. In our training process, we conduct several episodes, each consisting of multiple steps. In each step, we generate benign and malicious flows, and then we extract the environment's information in order to send it back to the RL agent, which calculates the new network's state and computes the reward/penalty to be awarded. At the end of each step, the action is adjusted based on the outcomes. Between episodes, we reset the environment to ensure that each new episode starts from a standard initial condition. This setup helps the RL agent learn from a variety of situations to improve its decision-making over time.

5.2 Results

Our primary objective is to demonstrate the effectiveness of our mitigation strategy in ensuring network performance under malicious behavior and in minimizing both network delay and negative impact on benign hosts. In this section, we present and discuss the outcomes obtained after applying our approach on a sample network, the architecture of which is illustrated in Fig. 1. The network consists of three controlled switches and four hosts among which there is a server and an attacker. We trained our RL agent algorithm and ran the Mininet emulator inside a virtual machine running Ubuntu 20.04 system with six processors and 8 GB of RAM. The underlying physical host operates on Intel Core i7-12800H CPU with 20 logical processors.

To assess the impact, we conducted a thorough comparison of bandwidth variations according to the obtained RL solution for both malicious and benign hosts. Additionally, we analyzed the percentages of malicious and benign packets that successfully reached the server across various scenarios represented by variations of DDoS attacks and random network configurations in terms of the server's and attacker's positions and varied volumes of packets generated by both benign and malicious hosts. These comparisons provide valuable insights into the robustness of our mitigation approach in preserving network availability under diverse attack conditions.

The algorithm iterates through 50 episodes and a maximum of 100 steps per episode, working with an RL agent possessing a neural network (depicted in Fig. 3) made up of multiple layers, with an input layer composed of ($N_s = n \times m$) inputs, where n is the number of state metrics, and m is the number of network's

[1] https://scapy.net/
[2] https://github.com/MatrixTM/MHDDoS
[3] https://mininet.org/
[4] https://traffic.comics.unina.it/software/ITG/
[5] https://www.wireshark.org/docs/man-pages/tshark.html
[6] https://anonymous.4open.science/r/countermeasure-selection-with-RL-1st-approach-2024

hosts, along with three intermediate layers comprising a total of 238 neurons. This number has been tuned based on trial and error. The rectified linear unit (ReLU) activation function is used in the input, first, and second intermediate layers, while a sigmoid activation function is used in the last intermediate layer. The output layer uses a linear activation function and is composed of $(N_a = 2 \times N_h + 1)$ output actions, where N_h represents the number of hosts, 2 is the number of possible actions per host (increase & decrease), and 1 is the (do nothing) action.

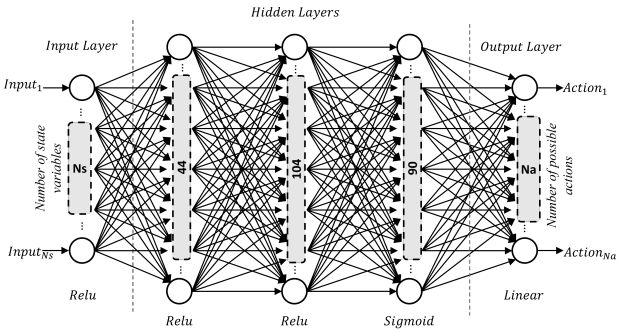


Figure 3: Neural Network Architecture.

Under various scenarios characterized by distinct types of attacks, the algorithm was able to converge with the same number of iterations for each of these types. However, it is noteworthy that variations were observed in the network performance metrics according to each attack type. The agent effectively mitigated the impact of malicious attacks targeting the server, as illustrated in Fig. 4. Simultaneously, it reduced the attacker's bandwidth by 96.7%, from 3 Mbps to 0.1 Mbps while enhancing it for normal hosts, leading to a noteworthy decrease in the rate of malicious traffic, and preventing network overload.

This impactful mitigation influenced the overall network performance, where the algorithm, as presented in Fig. 5, achieved an average delay of 8 ms across various DDoS attacks. This underscores the agent's effectiveness in mitigating attack impacts and optimizing overall network performance.

An evolution of the percentage of benign packets reaching the server relative to the number of total packets through various training stages is illustrated in Fig. 6. This percentage improved by 189%, increasing from 29.6% in the early training stages to 85.8% by the end of training, particularly during ICMP attacks. However, due to the severity of SYN attacks, where the server is overwhelmed by numerous half-open connections, the percentage of legitimate requests that can reach the server is significantly reduced. As a result, benign users could contribute only up to 39.4% of the traffic. This is a common challenge faced by many approaches dealing with SYN attacks [1, 17]. Notably, Fig. 7 indicates that ICMP attacks exhibited the lowest proportion of malicious packets, followed by UDP and SYN. Nevertheless, by the end of the training, malicious attacks diminished, and benign packets increased.

There was a consistent increase in the total reward over episodes during training, as depicted in Fig. 8. The agent reached the maximum reward and achieved better reward values for ICMP than UDP and SYN attacks.

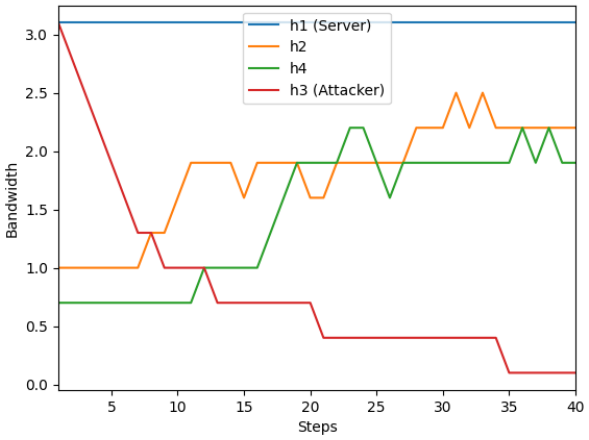


Figure 4: Bandwidth variations for benign and malicious hosts in a post-training scenario.

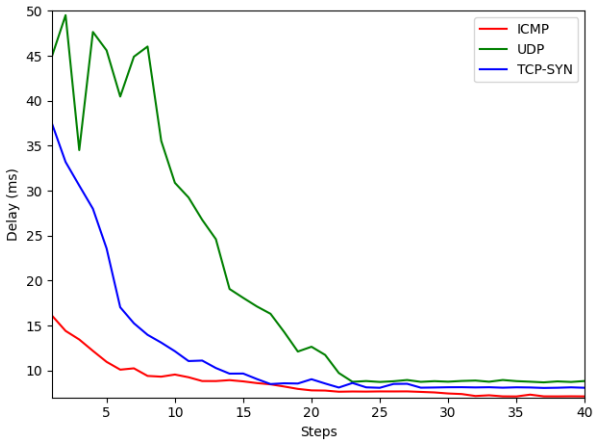


Figure 5: Variations of network's delay under different attack types.

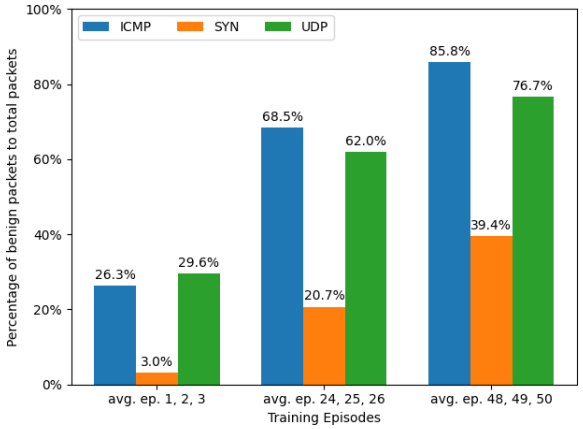


Figure 6: Percentage of benign packets reaching the server during different attack types.

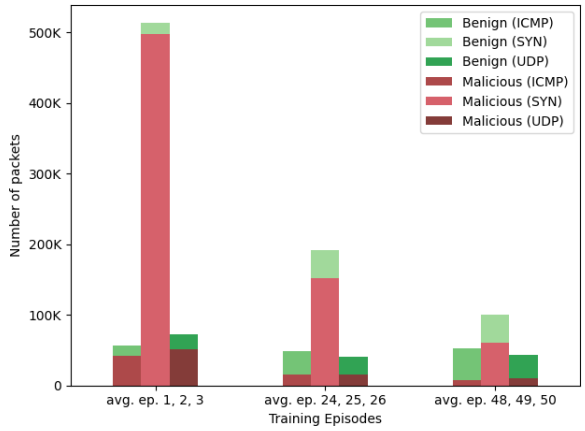


Figure 7: Total number of benign and malicious packets during different attack types.

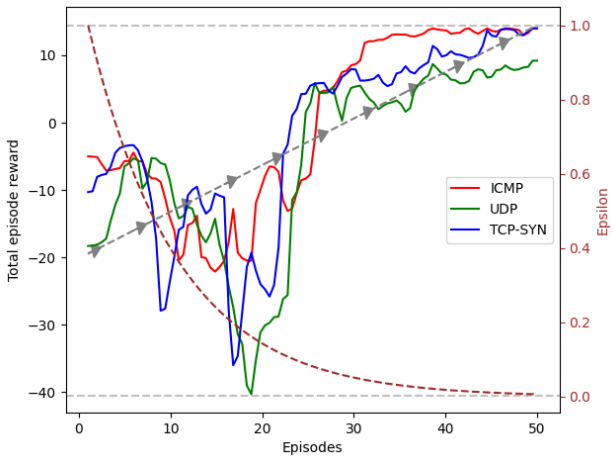


Figure 8: Cumulative agent's rewards per episodes.

5.3 Qualitative comparison

In this section, we present a qualitative comparison with state-of-the-art methods presented in section 2.1. We opted for qualitative rather than quantitative comparisons due to the significant challenges in directly comparing our approach with others for various reasons: handling different attack types [4, 22, 23], dependency on datasets rather than real traffic generation methods [15, 21], absence of source code and artifacts [13, 15, 21–23], or using different metrics for performance evaluation [15, 21]. Table 2 outlines key characteristics of our approach and how it differs from existing solutions.

Table 2: Qualititive comparison

Aspect	Our Proposed DDQN Approach	State-of-the-Art Approaches
Algorithm	Double Deep Q-Network (DDQN).	Various algorithms like Long Short-Term Memory (LSTM), Fuzzy Logic, DQN, Proximal Policy Optimization (PPO), etc.[15, 23].
Working Environment	Mininet with Customized Controller.	SDN Floodlight, ONOS, and Ryu controller [13, 15, 22].
Traffic nature	Tested with real-life traffic generators and varied traffic volumes for benign and malicious flows.	Often tested with predefined datasets [15, 21, 22].
Remediation strategy	Mitigate the malicious attacks on the network while enhancing overall performance.	Concentrate only on detecting and mitigating malicious attacks [15, 21].
Performance	Varies across different attack types (e.g., less effective against SYN attacks compared to other types).	Can be prone to over-blocking legitimate traffic (e.g., LSTM-Fuzzy) [15].
Evaluation Metrics	Packet loss, Network delay, Bandwidth usage, Number of benign packets reaching the server.	False positive rate, Resource consumption, Throughput and latency [15, 21].

5.4 Discussion

Our proposed mitigation strategy leverages a customized controller, making it adaptable to various SDN controllers and deployment scenarios. It also integrates a realistic attacker model with intelligent and dynamically positioned servers and attackers, mirroring real-world scenarios.

In spite of our efforts to minimize the negative impact and mitigate the influence of malicious attacks on the network, Fig. 7 illustrates our ability to identify and manage traffic, even during a SYN attack. Furthermore, when subjected to a SYN attack, as depicted in Fig. 6, our system effectively delivered a significant volume of benign packets. However, this volume can be further enhanced with additional refinements.

Finally, the time required for training can increase with larger topologies, which highlights the potential for improving our system's adaptability to complex topologies. Our proposed solution offers three levels of reusability. At the first level, it allows for easy adjustments of attack and server positions, attack types, and traffic rates with minimal code changes without altering the neural network's configuration. The second level supports the use of an external controller and the creation of new flooding DDoS attacks with additional code adjustments, maintaining the neural network's architecture. The third level enables expanding the network topology with new hosts, requiring modifications in the neural network, such as increasing neuron count in hidden layers and adjusting DDQN algorithm parameters to ensure optimal performance.

6 Conclusion and future work

We presented an RL-based technique for DDoS attack mitigation in SDNs using the DDQN algorithm that learns how to select the most appropriate remediation actions depending on the network state. We have also shown how our approach can effectively adapt to different attack scenarios and reduce the attack severity and network downtime while overcoming the negative side effects related to traditional remediation techniques. However, the application of RL

for implementing defensive measures in SDNs is still in its early research phase, leaving significant potential for further advancements and enhancements [10].

In future work, we plan to reimplement some of the most related works to experimentally quantify the performance difference with our approach. Additionally, we intend to demonstrate the scalability of the proposed methodology by integrating more complex network topologies.

Acknowledgments

This work is funded by the French National Research Agency through the GRIFIN project (ANR-20- CE39-0011).

References

[1] Basheer Al-Duwairi, Eslam Al-Quraan, and Yazeed AbdelQader. 2020. ISDSDN: mitigating SYN flood attacks in software defined networks. *Journal of Network and Systems Management* 28, 4 (2020), 1366–1390.

[2] Abdullah Algarni and Vijey Thayananthan. 2021. Improvement of 5G transportation services with SDN-based security solutions and beyond 5G. *Electronics* 10, 20 (2021), 2490.

[3] Sabah Alzahrani and Liang Hong. 2018. Generation of DDoS attack dataset for effective IDS development and evaluation. *Journal of Information Security* 9, 4 (2018), 225–241.

[4] Arti Bandhana, Ondřej Lukáš, Sebastian Garcia, and Tomáš Kroupa. 2023. Catch Me If You Can: Improving Adversaries in Cyber-Security With Q-Learning Algorithms. *arXiv preprint arXiv:2302.03768* (2023).

[5] Sunny Behal and Krishan Kumar. 2017. Characterization and Comparison of DDoS Attack Tools and Traffic Generators: A Review. *Int. J. Netw. Secur.* 19, 3 (2017), 383–393.

[6] Canadian Institute for Cybersecurity. 2020. CIC Flow Meter. https://github.com/CanadianInstituteForCybersecurity/CICFlowMeter.

[7] Rajasekhar Chaganti, Bharat Bhushan, and Vinayakumar Ravi. 2023. A survey on Blockchain solutions in DDoS attacks mitigation: Techniques, open challenges and future directions. *Computer Communications* 197 (2023), 96–112.

[8] Ammarah Cheema, Moeenuddin Tariq, Adnan Hafiz, Muhammad Murad Khan, Fahad Ahmad, and Muhammad Anwar. 2022. Prevention Techniques against Distributed Denial of Service Attacks in Heterogeneous Networks: A Systematic Review. *Security and Communication Networks* 2022 (2022), 1–15.

[9] Cloudflare. 2023. *DDoS Threat Report 2023 Q2.* Cloudflare Blog. https://blog.cloudflare.com/ddos-threat-report-2023-q2/

[10] José Álvaro Fernández-Carrasco, Lander Segurola-Gil, Francesco Zola, and Raul Orduna-Urrutia. 2022. Security and 5G: Attack mitigation using Reinforcement Learning in SDN networks. In *2022 IEEE Future Networks World Forum (FNWF).* IEEE, 622–627.

[11] Hisham A Kholidy, Andrew Karam, James L Sidoran, and Mohammad A Rahman. 2021. 5g core security in edge networks: A vulnerability assessment approach. In *2021 IEEE Symposium on Computers and Communications (ISCC).* IEEE, 1–6.

[12] Marius Letourneau, Guillaume Doyen, Rémi Cogranne, and Bertrand Mathieu. 2023. A Comprehensive Characterization of Threats Targeting Low-Latency Services: The Case of L4S. *Journal of Network and Systems Management* 31, 1 (2023), 19.

[13] Yandong Liu, Mianxiong Dong, Kaoru Ota, Jianhua Li, and Jun Wu. 2018. Deep reinforcement learning based smart mitigation of DDoS flooding in software-defined networks. In *2018 IEEE 23rd international workshop on computer aided modeling and design of communication links and networks (CAMAD).* IEEE, 1–6.

[14] Thanh Thi Nguyen and Vijay Janapa Reddi. 2021. Deep reinforcement learning for cyber security. *IEEE Transactions on Neural Networks and Learning Systems* (2021).

[15] Matheus P Novaes, Luiz F Carvalho, Jaime Lloret, and Mario Lemes Proenca. 2020. Long short-term memory and fuzzy logic for anomaly detection and mitigation in software-defined network environment. *IEEE Access* 8 (2020), 83765–83781.

[16] Radware. 2023. *Radware Full Year 2022 Report: Malicious DDoS Attacks.* https://www.radware.com/newsevents/pressreleases/2023/radware-full-year-2022-report-malicious-ddos-attacks/

[17] Avtar Singh, Harpreet Kaur, and Navjot Kaur. 2024. A novel DDoS detection and mitigation technique using hybrid machine learning model and redirect illegitimate traffic in SDN network. *Cluster Computing* 27, 3 (2024), 3537–3557.

[18] Ismael Amezcua Valdovinos, Jesús Arturo Pérez-Díaz, Kim-Kwang Raymond Choo, and Juan Felipe Botero. 2021. Emerging DDoS attack detection and mitigation strategies in software-defined networks: Taxonomy, challenges and future directions. *Journal of Network and Computer Applications* 187 (2021), 103093.

[19] Hado Van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 30.

[20] Alan Tamer Vasques and João JC Gondim. 2019. Amplified reflection ddos attacks over iot mirrors: A saturation analysis. In *2019 Workshop on Communication Networks and Power Systems (WCNPS).* IEEE, 1–6.

[21] Jin Wang and Liping Wang. 2022. SDN-Defend: A Lightweight Online Attack Detection and Mitigation System for DDoS Attacks in SDN. *Sensors* 22, 21 (2022), 8287.

[22] Noe M Yungaicela-Naula, Cesar Vargas-Rosales, Jesús Arturo Pérez-Díaz, and Diego Fernando Carrera. 2022. A flexible SDN-based framework for slow-rate DDoS attack mitigation by using deep reinforcement learning. *Journal of Network and Computer Applications* 205 (2022), 103444.

[23] Mikhail Zolotukhin, Sanjay Kumar, and Timo Hämäläinen. 2020. Reinforcement learning for attack mitigation in SDN-enabled networks. In *2020 6th IEEE conference on network softwarization (NetSoft).* IEEE, 282–286.